

Repo Overview

May 2, 2026

Artifact Repository Structure

The source code, cryptographic libraries, and experimental benchmarks for the BioPAKE protocol are organized within the repository as follows:

`precomputescalmult/`

The C library binding which performs our Elliptic Curve optimizations, including pre-computation of scalar multiplication tables.

`biopake_bulletproofs/`

Our interface to the core Rust library which implements the non-interactive zero-knowledge proofs (Bulletproofs). This module handles the generation and verification of the range proofs required during the interactive authentication phase.

`BioPAKE.ipynb`

The primary Jupyter Notebook which contains our prototype BioPAKE protocol implementation, together with the end-to-end experimental pipeline for testing this prototype. This notebook contains (1) the script that allows for testing our prototype and replicating our results, i.e. Experiment #1, Section 7.2 (2) the script that also generates cryptographic micro-benchmarks (both scripts perform tests across various parameter sweeps). Finally, (3) it contains a script that performs the Angular Geometric Analysis, i.e. Experiment #2, Section 7.3.

`RAW_Results.xlsx`

The Excel spreadsheet which contains the results of our tests, including Experiment #1 and #2 described in Section 7, including the results shown in Figures 2 and 3 in Section 7.

`bsp.py`

The Python module with the subroutines (called from `BioPAKE.ipynb`) which implement Cosine Locality-Sensitive Hash (CosineLSH). This script manages the generation of random hyperplanes, the extraction of binary representations from floating-point biometric vectors, and the filtering out of the ‘ignored hyperplanes’.

`extract_embeddings.py`

The Python script which creates the JSON file from the VGGFACE2 data
(which can be download from www.kaggle.com/datasets/hearfool/vggface2)

Dependency Guide

This section outlines the system prerequisites of the primary experimental pipeline (BioPAKE.ipynb), and the execution instructions necessary to reproduce the empirical results presented in this paper.

System Dependencies & Basic Prerequisites

Before executing the notebook, the evaluation environment must be configured with the following dependencies:

- **Python 3.8+** with an active Jupyter Notebook or JupyterLab environment.
- **Libsodium:** Required for the foundational Elliptic Curve operations.
 - *Ubuntu/Debian:* `sudo apt-get install libsodium-dev`
 - *macOS:* `brew install libsodium`
- **Python Packages:** `numpy`, `pandas`, `cryptography`, `pycryptodome`, and `jupyter`.

0.0.1 Rust Library Compilation:

The BioPAKE protocol relies on a custom Rust implementation of non-interactive zero-knowledge range proofs. This library is exposed to the Python binding defined in the `pyproject.toml` file in the `python_bulletproofs` folder. Before running the Jupyter notebook, this library must be compiled and injected into your Python environment.

1. **Install the Rust Toolchain:** If Rust is not already installed on your system, install it via `rustup` (the official installer):

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2. **Navigate to the Source Directory:** Open your terminal and change directories into the folder containing the `Cargo.toml` and `pyproject.toml` files:

```
cd /path/to/biopake_bulletproofs
```

3. **Install the Build System:** We recommend building the bindings within an active Python virtual environment (e.g., `venv` or `conda`). Install the Maturin build system via `pip`:

```
pip install maturin
```

4. **Compile and Install:** Execute Maturin to compile the Rust code with maximum release optimizations (`--release`) and immediately install the resulting module into your active Python environment. *Note: Do not omit the release flag, as unoptimized debug builds of cryptographic proofs will execute prohibitively slowly.*

```
maturin develop --release
```

Alternative Standard Build: You may also trigger the build using standard Python packaging commands:

```
pip install .
```

0.0.2 Dataset Preparation:

The notebook relies on the JSON file for VGGFACE2 dataset. To prepare such JSON file, one should run the `extraction_embedding.py`. The script requires the `insightface` library, which should be installed using the following command:

```
pip install insightface onnxruntime opencv-python numpy
```

In order for the script to find the downloaded dataset, one need to replace the variable

```
DATASET_ROOT = "/path/to/vggface2/train"
```

to the actual path for the `vggface2/train` dataset. After running the script, the JSON file will have the name

```
"data_train_resnet50.json"
```

0.0.3 Compiling the EC Hardware Acceleration (C Bindings)

Before executing the protocol, this C module must be compiled into a Shared Object (`.so`) library so it can be loaded via Python's `ctypes` interface.

Execute the Build Script: Navigate to directory `precomputescalmult` containing the C source code and the provided `build.sh` script. Make the script executable and run it:

```
chmod +x build.sh
./build.sh
```

0.0.4 Jupyter Notebook Setup

Navigate to the Jupyter Notebook `BioPAKE.ipynb` and run through section 1 to section 4. These sections generate no output, but they define the variables, functions, and classes used by the experiments.

0.1 Replicating Experiment #1

First Run section 6 of the jupyter notebook to initialize the experiment routine. Then run section 7 of the Jupyter Notebook `BioPAKE.ipynb`. This will write a csv file `pake_profiling_resnet50_big.csv`, which contains the full result for experiment 1.

0.2 Replicating Experiment #2

To obtain the result for our Experiment #2, run the section 8 of the Jupyter Notebook.